Minimum Viable Specification for a Worldwide Agentic Web

Draft for web implementers and agent system builders

MVP Profile 0.1 | 3 November 2026 | Cognizant AI Lab

Purpose This document defines the minimum interoperable behavior needed to publish and run portable multi-agent systems on the existing web. A conforming agent site publishes a data-only network definition. A standard web browser runtime runs that network with the site's default inference configuration or, when offered, a visitor-selected router and credential. With the visitor's approval, one agent site can load a second site's published agents and run them in the caller's browser.

The profile uses ordinary HTTPS hosting, browser security boundaries, CORS, JavaScript modules, JSON Schema, and MCP. It does not require a new transport, a central registry, or a server-side agent runtime. The goal is a small target that independent teams can implement on the existing web and extend through compatible services, tools, and clients.

Scope and conformance

This profile defines three core tenets and the publication, privacy, and safety rules that make them interoperable. An implementation may use different internal libraries or agent frameworks. It conforms when it produces and consumes the same public artifacts and observable behaviors.

Version 0.1 This draft records the minimum profile implemented by a reference runtime.

The key words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are to be interpreted as described in RFC 2119 and RFC 8174.

Capability	Minimum requirement	Interoperability result
Portable network	Neuro SAN HOCON data model with a resolved, data-only web artifact	The same multi-agent graph can move between publishers and runtimes
Browser execution	JavaScript orchestration, validation, tool calls, and linked-site loading	A normal browser is the agent container
Pluggable inference	Site-provided default router and model with optional visitor BYOK	Inference funding and deployment can change without changing the agent graph
Web publication	Manifest, network definition, routing and peer-policy metadata, and CORS	A caller can locate and validate a peer without a central registry
Trust boundary	Consent, browser and CORS boundaries, private-context filtering, and execution limits	A linked site cannot silently acquire the user's router credential or unlimited execution

Intended use and boundaries

Version 0.1 targets open, portable agent networks that a normal web page can publish and a visitor's browser can run. The published HOCON, including agent instructions, is public by design. This lets users and implementers inspect behavior, compare implementations, test portability, and build compatible runtimes. Publisher-funded and user-funded model access both fit this profile. A publisher that needs confidential instructions, controlled data, or secure execution can keep that functionality server-side and expose a typed capability through authenticated MCP, a site action, or a hosted agent protocol. Authentication, billing, and service-level policies for those services are outside the minimum requirements.

- **Tab-scoped execution:** The browser runtime operates for the lifetime of the page. Scheduled, event-driven, background, or durable tasks require a hosted runtime or another service.

- **Direct linking:** A caller uses a published URL or a named link supplied by the publisher or user.
- **Publisher operations:** Analytics, rate limits, monetization, liability allocation, and commercial terms are publisher concerns or future metadata, not interoperability requirements.

Terms

- **Agent network:** A named graph of agents, their instructions, invocation schemas, and available agents or tools.
- **Agent site:** A web origin or subpath that publishes one or more agent networks and the files needed to run them.
- **Caller site:** The site and browser runtime handling the user's current session.
- **Peer site:** A second agent site whose network is loaded by the caller.
- **Front agent:** The public entry point for an agent network. Other published agents support the network but are not its external entry point.
- **LLM router:** A site-provided or user-selected, OpenAI-compatible endpoint that routes model requests to one or more models.

Tenet one: common multi agent representation

The canonical representation MUST follow the Neuro SAN Agent Network HOCON File Reference. HOCON is the canonical authoring and exchange representation. Because JSON is a valid HOCON serialization, a site MAY publish an equivalent, fully resolved JSON document for browser loading. The published form MUST preserve the Neuro SAN data model and MUST NOT depend on local includes, environment substitutions, or publisher-only file paths.

Minimum network content

- **Identity and metadata:** A stable network name, a short description, and the name of its front agent.
- **Agent definitions:** Each agent has a unique name, a function description, instructions when it uses an LLM, and a list of agents or tools it may call.
- **Invocation schemas:** Callable agents and tools expose JSON Schema-compatible inputs. Structured outputs SHOULD also declare schemas.
- **Model settings:** Model names and generation settings are hints, and the caller's active router and model policy take precedence when a peer runs in the browser. A compatible extension may let a publisher declare tested or permitted model lists, model families, or capability ranges and let the runtime warn or refuse when the selected model falls outside that policy.
- **Private context rules:** Private data is separate from chat content and is deny-by-default across agent-site boundaries. A network must explicitly name fields that may pass downstream or upstream.
- **External capabilities:** An agent may reference another published agent-network URL or an MCP server. An MCP descriptor may restrict the tool names exposed to the agent. A runtime MAY also expose a catalog of registered site actions. An agent may propose a call to an exact catalog entry; the runtime validates the arguments, obtains any required user confirmation, and dispatches the call to the site-owned handler.

Portable data profile

The minimum interoperable network definition is declarative data. Its public instructions and schemas are an intentional interoperability surface, not a confidentiality mechanism; reuse remains subject to the publisher's license and terms. No coded-tool bundle is required. A build step may resolve HOCON includes, reusable definitions, Python coded tools, or framework-specific configuration into the published network. Confidential or code-backed capabilities can remain server-side and be exposed through MCP, a local site action, a hosted agent protocol, or the optional coded-tool mechanism implemented by the reference runtime.

```
metadata {
  description = "Local services guide"
}
tools = [
  {
    name = "guide"
    function.description = "Finds the right local service"
    instructions = "Understand the request and delegate it"
    tools = ["lodging", "https://peer.example/transit"]
  },
  {
    name = "lodging"
    function.description = "Answers lodging questions"
    instructions = "Use only grounded lodging information"
    tools = [{ url = "https://tools.example/mcp", tools = ["search"] }]
  }
]
```

Other agent frameworks can map into this representation. Google ADK names, descriptions, instructions, sub-agents, tools, models, and input or output schemas map to core HOCON constructs. Framework-specific workflows, lifecycle hooks, state, or custom runtime behavior may require an adapter or may not be portable. A converter or Skill file may automate the mapping; conformance is determined by the emitted HOCON profile, not by the source framework. See the Google ADK agent configuration documentation and a reference implementation for a convertor.

Tenet two: browser execution and site composition

A conforming runtime MUST be usable from a normal web page and MUST execute the published graph in JavaScript. The nsDeploy browser runtime is the reference implementation. It parses and validates a network, runs the front agent and internal agents, translates callable capabilities into model-visible function schemas, dispatches approved calls, and returns structured results to the parent agent.

Minimum runtime behavior

- **Parse and validate:** Reject malformed networks, duplicate or missing agent references, an invalid front agent, and unsupported executable definitions before any model call.
- **Run the agent loop:** Support model streaming, tool calls, agent-to-agent delegation, error propagation, and bounded conversation history.
- **Use grounded capabilities:** Support MCP as the baseline interface for tools and data. Ask for user approval before discovering or calling a third-party MCP service.
- **Handle local site actions:** Support same-site registered actions described by name, purpose, JSON Schema, effect, and handler. The runtime MUST validate arguments and show the user the destination and arguments before a write or irreversible operation. Cross-site action transport is an extension.
- **Report outcomes accurately:** Distinguish a returned HTTP or tool response from a verified business outcome. Timeouts and disconnects after a write begin have unknown outcome and MUST NOT be retried automatically.
- **Bound execution:** Enforce cycle detection and finite limits on composition depth, agent calls, remote calls, concurrency, and elapsed time. The reference runtime permits at most five composition levels, four remote calls, one call per peer, 24 agent invocations, and 90 seconds per query; a coded tool has a 15-second default timeout.

Protocol for loading a peer agent site

This is the minimum agent-site-to-agent-site protocol. It composes agent definitions, not remote execution services. The peer's agents run in the caller's browser.

1. **Reference:** The caller network identifies a peer as an HTTP or HTTPS URL whose final path component names the peer network or resolves a named link to that URL. The peer-network URL omits a trailing slash. The runtime removes the final component to derive the site base, so https://peer.example/agents/transit resolves its manifest at https://peer.example/agents/agent-manifest.json. HTTPS is required outside local development.

2. **Consent:** Before contacting a third-party origin, the interactive runtime shows the peer origin, the intended request summary, and an estimated token count or cost when available, then obtains the user's approval. The user may allow once, always allow that origin for the current session, or decline.
3. **Manifest discovery:** The runtime fetches `agent-manifest.json` from the peer site base and selects the exact requested network. It MUST NOT silently fall back to another published network.
4. **Definition fetch:** The runtime resolves the manifest's hocon value against the peer site base, fetches the referenced network definition, and validates it before execution. The value may be a relative path or an absolute HTTP or HTTPS URL. Version 0.1 follows normal browser fetch redirects and relies on browser CORS enforcement; it does not require the final network-definition URL to remain on the manifest's origin.
5. **Local execution:** The caller instantiates the peer network in the caller's runtime. The peer inherits the caller's active LLM router, model policy, and execution limits. When a user key is active, it is sent only to that router and never to the peer site.
6. **Result return:** The peer front agent's answer and structured output return through the caller's normal tool-result path. Private context remains filtered unless the caller explicitly allows named fields.
7. **Per hop policy:** Every additional peer or MCP hop repeats the origin-policy and consent checks. The runtime stops cycles and requests that exceed its budgets.

MVP boundary Loading a peer's declarative agents is required. A conforming site does not need coded tools, but the reference runtime may load a manifest-declared bundle under the Worker restrictions described above. Cross-site action execution is an extension: action code remains on its owning origin and uses an origin-checked bridge with a second confirmation for remote writes.

Minimum public files

Resource	Required content	Hosting rule
agent-manifest.json	One or more networks with name, front_man, and hocon; top-site LLM router metadata; peer policy; optional brand, MCP, linking, deployment, action, theme, and coded-tool metadata	Public GET from the derived site base; CORS enabled for callers
Network definition	Resolved Neuro SAN HOCON data model, commonly serialized as <network>.network.json	Relative or absolute HTTP(S) location; public GET with CORS when cross-origin
links.json	Publisher-owned default bindings from named references to peer-network URLs	Public GET; browser-local user overrides are stored separately by the caller origin
Optional coded-tool bundle	Self-contained JavaScript module and optional SRI digest declared by the manifest	Public GET; executed only in the restricted Worker
.well-known/nsdeploy files	Optional site identity and exported-network discovery metadata	Published by the current generator but not read by the version 0.1 execution loader

The publisher owns the public links.json defaults. When a user supplies a missing binding, the reference runtime stores that override in origin-scoped browser storage and may persist it through a dynamic site endpoint; a static deployment does not rewrite the published file. Only public agent artifacts need cross-origin CORS. Action modules, authentication endpoints, and business operations remain on their owning origin and keep their normal browser security policy. Production deployments MUST use HTTPS; loopback HTTP is acceptable for local development.

The optional nsDeploy .well-known files provide discovery metadata only. They are not an alternate execution path. A site may separately publish an A2A Agent Card at /.well-known/agent-card.json for A2A interoperability; the version 0.1 nsDeploy loader does not consume that card.

Tenet three: pluggable inference

The agent graph is independent of the service that performs inference. A site MAY publish a default router and model, including a fixed endpoint operated or funded by the publisher. It MAY also offer bring your own key (BYOK) so the visitor supplies a router, model, and credential. The reference implementation supports both modes through the nsDeploy browser engine, its BYOK profiles, and its OpenAI-compatible LLM client.

- **Default inference:** A site MAY publish a default router and model that work without a visitor-supplied key. The site may absorb inference cost, authenticate the browser through its own session, or route to infrastructure it controls. The runtime MUST identify mock execution and MUST NOT describe it as real model output.
- **Optional BYOK:** A runtime can conform without offering BYOK when the published default inference path runs the network. When BYOK is offered, the user MUST be able to select or enter the permitted router base URL, model, and credential, subject to any supported site model policy.
- **Credential handling:** When BYOK is active, the key remains in the caller's browser and is attached only to model requests sent to the selected router. A conforming runtime MUST NOT place it in URLs, manifests, HOCON, prompts, action arguments, traces, analytics, or peer-site requests.
- **Router-only disclosure:** The top site selects the active default or BYOK router. A loaded peer inherits that router; its manifest cannot replace the router or receive a user key. Router metadata in a peer manifest is therefore informational during composition.
- **BYOK proxy option:** A dynamic site MAY expose a same-origin forwarding endpoint when the visitor's chosen router does not allow browser CORS. In BYOK mode, the proxy forwards the visitor's authorization header and MUST NOT substitute a host-owned provider key. A separate default inference endpoint may use publisher-controlled credentials on its server.
- **Static BYOK option:** A static bundle MAY call an OpenAI-compatible router directly when that router permits browser CORS. When the selected router requires a key, a missing key must stop the model call with a clear setup error.
- **Reference storage:** When BYOK is used, the reference runtime stores profiles, including credentials, in origin-scoped localStorage. On loopback hosts only, it may mirror the profile to a host cookie so settings follow local ports. The user can delete stored profiles; shared devices require particular care.
- **Threat boundary:** BYOK can remove routine credential custody and inference cost from a publisher's server, but it does not isolate the credential from other JavaScript executing on the caller origin. Publishers SHOULD minimize third-party scripts and deploy a restrictive Content Security Policy.

Cross cutting security and privacy requirements

- **Untrusted instructions:** Peer definitions are untrusted model input. Declarative data prevents ambient code execution, but instructions can still attempt prompt injection, induce disclosure, or request unsafe tool calls. Validation and the coded-tool sandbox do not make a peer trustworthy.
- **Optional peer code:** A coded-tool bundle is fetched and executed only when the manifest declares it. The restricted Worker is the security boundary for that code. Integrity is checked when the publisher supplies a supported SRI digest; version 0.1 does not require the digest.
- **Explicit destinations:** The consent interface identifies the peer or MCP host before the initial third-party contact and shows the request text or tool arguments in summarized form. Absolute manifest references and subsequent redirects still follow ordinary browser fetch and CORS behavior.
- **Consent before contact:** Interactive approval precedes third-party manifest fetches, MCP discovery, and MCP calls. A remembered approval is scoped to the destination origin for the current session. Consent records a user's decision; it is not evidence that peer instructions or model-generated descriptions are safe.
- **Private context by exception:** Credentials and private sly_data fields do not cross agent-site boundaries by default. Named allow rules control those fields. The delegated request text is sent to the peer front agent by design and is not protected by the sly_data filter.
- **MCP disclosure:** A third-party MCP service receives the model-generated tool arguments approved for its call. MCP authentication, OAuth, and service-specific credentials are separate from the LLM BYOK key and are not transferred automatically.
- **Fail closed:** Invalid definitions, unsupported schemas, missing network names, permission denial, composition cycles, and exceeded limits stop the affected call and remain visible to the user.

- **User-owned writes:** The runtime, not the model, owns confirmation. The executed action, origin, arguments, request identifier, and receipt are bound to the reviewed proposal.
- **Observable execution:** Implementations SHOULD expose the selected router, contacted origins, agent and tool calls, consent decisions, errors, and action receipts without exposing secrets.

MVP interoperability checks

An implementation is interoperable when it can demonstrate all of the following without publisher-specific integration code.

- Publish a two-agent HOCON network and a manifest from an ordinary HTTPS web host.
- Load and run that network in a browser with the site's published default router and model; if BYOK is offered, also demonstrate the user-selected path.
- Call an internal agent and at least one grounded capability through MCP or a registered local site action.
- After user approval, load an exact named network from a second origin and run it locally in the caller's browser.
- When a user key is active, show that the second site never receives it and cannot change the caller's router.
- Show that private context is absent at the peer unless a named field is explicitly allowed.
- Stop an invalid network, a composition cycle, an over-budget request, and a denied origin with visible errors.

Extensions and community opportunities

These current prototypes and natural extensions are outside the minimum profile for portable agent data, browser execution, and pluggable inference.

- Transactions negotiation and arbitration Policy-bound mandates, sealed bids, deterministic negotiation, settlement controls, and auditable receipts through a trusted arbiter.
- Cross-site browser actions Site-owned action catalogs, origin-checked iframe or popup bridges, caller permissions, confirmation, idempotency, and unknown-outcome handling.
- Private stewards — Operator-hosted Neuro SAN networks that keep credentials and private policy on the steward’s server while loading and running public agent networks locally, with origin-level controls and an MCP compatibility fallback.
- CRUSE generative interfaces Agent-proposed forms, choices, reports, tables, charts, and action controls that are validated and rendered by the browser runtime.
- ANDe assisted HOCON authoring Natural-language creation and modification of HOCON networks through the Neuro SAN Agent Network Designer, usable through clients such as MAUI.
- MAUI client A web client for hosted Neuro SAN systems and agent-network authoring experiences.
- Framework converters Skills and tools that translate Google ADK and other framework graphs into the portable HOCON profile and verify semantic equivalence.
- Authenticated MCP and hosted bridges OAuth, user or service credentials, hosted MCP gateways, and policy controls for private or commercial data sources.
- A2A discovery and hosted execution Standard Agent Cards, registries, authenticated remote agents, durable tasks, background work, scheduling, and operation beyond a browser tab's lifetime.
- Local and browser inference Local model servers and WebGPU or WebAssembly engines such as WebLLM could implement the inference interface without a provider key when they meet the site's model and tool policy.
- Versioning integrity and publisher terms Profile fields, hashes, caller pinning, signatures, licensing, attribution, and re-consent when published artifacts change.

Note that while examples of agentic sites linked from this spec tend to take the shape of chatbots, they need not necessarily be invoked in this manner. For example, one can conceive of agents auto redesigning a web page as it is being used, through analysis of implied context and user behavior.

Additional work includes model setting extensions, WebMCP integration, client-server agent extensions, registries and search, identity and reputation, model compatibility, capability negotiation, accessibility, conformance tests, consent policies, cost controls, caching, analytics, rate limits, agent harness extensions, looping and event-based agent implementations, sly_data inter-agent data transport, and profile governance.

Reference implementations and source material

- nsDeploy. Reference browser runtime for publishing, linked sites, pluggable inference, BYOK, MCP, and actions.
- Neuro SAN Agent Network HOCON File Reference. Normative common agent-network data model.
- Neuro SAN Studio. Agent Network Designer, examples, and hosted development tooling.
- A2A Protocol. Optional Agent Card discovery and hosted agent interoperability.
- Worldwide agentic web prototype demonstration. Current research prototype demonstration.